

Федеральное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: «Вычислительная математика»

Лабораторная работа №6

«Методы решения дифференциальных уравнений»

Вариант **9**

Выполнила:

Студентка группы Р3213 Пименова Екатерина Андреевна

Преподаватель:

Машина Екатерина Алексеевна

г. Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

ЦЕЛЬ РАБОТЫ	3
ВАРИАНТ 9.....	3
РАСЧЕТЫ ВРУЧНУЮ	4
ОПИСАНИЕ ИСХОДНЫХ ДАННЫХ.....	4
ПРИМЕР 1	4
ПРИМЕР 2	4
ПРИМЕР 3	7
КОД.....	5
MAIN.PY	5
APPROXIMATION_METHODS.PY.....	11
GRAPHICS_DRAWER.PY	14
TABLE_PRINTER.PY.....	18
РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ.....	10
ПРИМЕР 1	10
ПРИМЕР 2	12
ПРИМЕР 3.....	22
ВЫВОД	14

Цель работы

Цель работы – изучение дифференциальных уравнений, ручное решение и реализация методов решения на ЭВМ.

Вариант 9

1. Порядок выполнения работы

2. В программе численные методы решения обыкновенных дифференциальных уравнений (ОДУ) должен быть реализован в виде отдельного класса /метода/функции;
3. Пользователь выбирает ОДУ вида $y' = f(x, y)$ (не менее трех уравнений), из тех, которые предлагает программа;
4. Предусмотреть ввод исходных данных с клавиатуры: начальные условия $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h , точность ε ;
5. Для исследования использовать одношаговые методы и многошаговые методы (см. табл.1);
6. Составить таблицу приближенных значений интеграла дифференциального уравнения, удовлетворяющего начальным условиям, для всех методов, реализуемых в программе;
7. Для оценки точности одношаговых методов использовать правило Рунге;
8. Для оценки точности многошаговых методов использовать точное решение задачи: $\varepsilon = \max_{0 \leq i \leq n} |y_{i\text{точн}} - y_i|$;
9. Построить графики точного решения и полученного приближенного решения (разными цветами);
10. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных.
11. Проанализировать результаты работы программы.

Описание исходных данных

Пользователь вводит с клавиатуры:

- номер уравнения для решения;
- начальные значения X и Y ;
- конечные значения X и Y ;
- шаг интегрирования;
- желаемую точность.

пример 1

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/b
выберите уравнение для решения:
1: dy/dx = x + y
2: dy/dx = x^2 + y^2
3: dy/dx = 2x - y
введите номер уравнения (1, 2 или 3): 1
введите начальное значение x (x0): 0
введите начальное значение y (y0): 1
введите конечное значение x (xn): 1
введите шаг интегрирования (h): 0.2
введите желаемую точность (epsilon) 0.01
```

пример 2

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/b
выберите уравнение для решения:
1: dy/dx = x + y
2: dy/dx = x^2 + y^2
3: dy/dx = 2x - y
введите номер уравнения (1, 2 или 3): 2
введите начальное значение x (x0): 0
введите начальное значение y (y0): 1
введите конечное значение x (xn): 333
введите шаг интегрирования (h):
нееееет это все числа
```

КОД

main.py

```
import numpy as np
import matplotlib.pyplot as plt
import math

def euler_method(f, x0, y0, xn, h):
    x_values = [x0]
    y_values = [y0]
    x = x0
    y = y0
    while x < xn - 1e-8:
        y += h * f(x, y)
        x += h
        x_values.append(x)
        y_values.append(y)
    return x_values, y_values

def runge_kutta_4(f, x0, y0, xn, h):
    x_values = [x0]
    y_values = [y0]
    x = x0
    y = y0
    while x < xn - 1e-8:
        k1 = h * f(x, y)
        k2 = h * f(x + h / 2, y + k1 / 2)
        k3 = h * f(x + h / 2, y + k2 / 2)
        k4 = h * f(x + h, y + k3)
        y += (k1 + 2 * k2 + 2 * k3 + k4) / 6
        x += h
        x_values.append(x)
        y_values.append(y)
    return x_values, y_values

def adams_method(f, x0, y0, xn, h):
    x_rk, y_rk = runge_kutta_4(f, x0, y0, x0 + 3 * h, h)
    x_values = x_rk.copy()
    y_values = y_rk.copy()
    x = x0 + 3 * h
    while x < xn - 1e-8:
        f0 = f(x_values[-4], y_values[-4])
        f1 = f(x_values[-3], y_values[-3])
        f2 = f(x_values[-2], y_values[-2])
        f3 = f(x_values[-1], y_values[-1])

        # предиктор
        y_pred = y_values[-1] + h * (55 * f3 - 59 * f2 + 37 * f1 - 9 *
f0) / 24
        x_next = x + h
```

```

        f_pred = f(x_next, y_pred)

        # корректор
        y_corr = y_values[-1] + h * (9 * f_pred + 19 * f3 - 5 * f2 + f1)
/ 24
        x_values.append(x_next)
        y_values.append(y_corr)
        x = x_next
    return x_values, y_values

def get_equation(equation_choice):
    """Возвращает функцию правой части ОДУ по выбору пользователя"""
    equations = {
        '1': lambda x, y: x + y,
        '2': lambda x, y: x ** 2 + y ** 2,
        '3': lambda x, y: 2 * x - y
    }
    return equations.get(equation_choice, equations['1'])

def get_exact_solution(equation_choice, x0, y0):
    solutions = {
        '1': lambda x: -x - 1 + (y0 + x0 + 1) * math.exp(x - x0),
        '3': lambda x: 2 * x - 2 + (y0 - 2 * x0 + 2) * math.exp(-(x -
x0))
    }
    return solutions.get(equation_choice)

def runge_rule_error(y_h, y_h2, p):
    return abs(y_h - y_h2) / (2 ** p - 1)

def calculate_errors(results, exact_solution_func, h, equation_choice,
x0, y0, xn):
    errors = {}
    for method_name, (x_values, y_values) in results.items():
        if method_name in ['euler', 'runge_kutta4']:
            h_half = h / 2
            if method_name == 'euler':
                _, y_values_half =
euler_method(get_equation(equation_choice), x0, y0, xn, h_half)
                order = 1
            elif method_name == 'runge_kutta4':
                _, y_values_half =
runge_kutta_4(get_equation(equation_choice), x0, y0, xn, h_half)
                order = 4
            # Находим ближайшие точки для сравнения (последняя точка при
шаге h и соответствующая при шаге h/2)
            y_h_end = y_values[-1]
            y_h2_end = y_values_half[int(len(y_values_half) * h / (xn -
x0))] if y_values_half else float('nan')
            errors[method_name] = runge_rule_error(y_h_end, y_h2_end,
order)

```

```

        elif method_name == 'adams':
            if exact_solution_func:
                exact_values = [exact_solution_func(x) for x in
x_values]
                errors[method_name] = max(abs(y - exact) for y, exact in
zip(y_values, exact_values))
            else:
                errors[method_name] = float('nan')
        else:
            errors[method_name] = float('nan')
    return errors

def display_errors(errors):
    print("\nпогрешности методов:")
    for method_name, error in errors.items():
        print(f"{method_name}: {error:.6f}")

def print_results_table(euler_results, rk4_results, adams_results,
exact_values=None):
    x_euler, y_euler = euler_results
    _, y_rk4 = rk4_results
    _, y_adams = adams_results

    print("\n{:<10} {:<15} {:<15} {:<15} {:<15}".format("X", "euler",
"Runge-Kutta4", "adams", "Exact"))
    print("-" * 70)
    for i, x in enumerate(x_euler):
        exact = f"{exact_values[i]:.6f}" if exact_values else "N/A"
        print("{:<10.4f} {:<15.6f} {:<15.6f} {:<15.6f} {:<15}".format(x,
y_euler[i], y_rk4[i], y_adams[i], exact))

def plot_results(euler_results, rk4_results, adams_results,
exact_solution_func=None):
    x_euler, y_euler = euler_results
    x_rk4, y_rk4 = rk4_results
    x_adams, y_adams = adams_results

    plt.figure(figsize=(10, 6))
    plt.plot(x_euler, y_euler, label='euler', linestyle='--',
color="cyan")
    plt.plot(x_rk4, y_rk4, label='runge-kutta 4', linestyle='-.',
color="deeppink")
    plt.plot(x_adams, y_adams, label='adams', linestyle=':')
    if exact_solution_func:
        x_exact = np.linspace(min(x_euler), max(x_euler), 500)
        y_exact = [exact_solution_func(x) for x in x_exact]
        plt.plot(x_exact, y_exact, label='Exact', color='black')

    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('сравнение численных методов решения дифф уравнений')
    plt.legend()

```

```

plt.grid(True)
plt.show()

def get_input():
    """Получает входные данные от пользователя"""
    print("выберите уравнение для решения:")
    print("1: dy/dx = x + y")
    print("2: dy/dx = x^2 + y^2")
    print("3: dy/dx = 2x - y")
    equation_choice = input("введите номер уравнения (1, 2 или 3): ")

    try:
        x0 = float(input("введите начальное значение x (x0): "))
        y0 = float(input("введите начальное значение y (y0): "))
        xn = float(input("введите конечное значение x (xn): "))
        h = float(input("введите шаг интегрирования (h): "))
        epsilon = float(input("введите желаемую точность (epsilon) "))
    except ValueError:
        print("неееееет это все числа")
        return None, None, None, None, None, None, None

    exact_solution_func = get_exact_solution(equation_choice, x0, y0)
    return equation_choice, x0, y0, xn, h, epsilon, exact_solution_func

def solve_and_evaluate(equation_choice, x0, y0, xn, h, epsilon,
exact_solution_func=None):
    """Решает ОДУ различными методами, выводит результаты и строит
графики"""
    f = get_equation(equation_choice)
    results = {
        'euler': euler_method(f, x0, y0, xn, h),
        'runge_kutta4': runge_kutta_4(f, x0, y0, xn, h),
        'adams': adams_method(f, x0, y0, xn, h)
    }

    errors = calculate_errors(results, exact_solution_func, h,
equation_choice, x0, y0, xn)
    display_errors(errors)

    exact_values = [exact_solution_func(x) for x in results['euler'][0]]
    if exact_solution_func else None
    print_results_table(results['euler'], results['runge_kutta4'],
results['adams'], exact_values)

    plot_results(results['euler'], results['runge_kutta4'],
results['adams'], exact_solution_func)

def main():
    """Главная функция, запускающая процесс решения и сравнения
методов"""
    inputs = get_input()
    if inputs:

```



```
equation_choice, x0, y0, xn, h, epsilon, exact_solution_func =
inputs
    solve_and_evaluate(equation_choice, x0, y0, xn, h, epsilon,
exact_solution_func)

if __name__ == "__main__":
    main()
```

РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

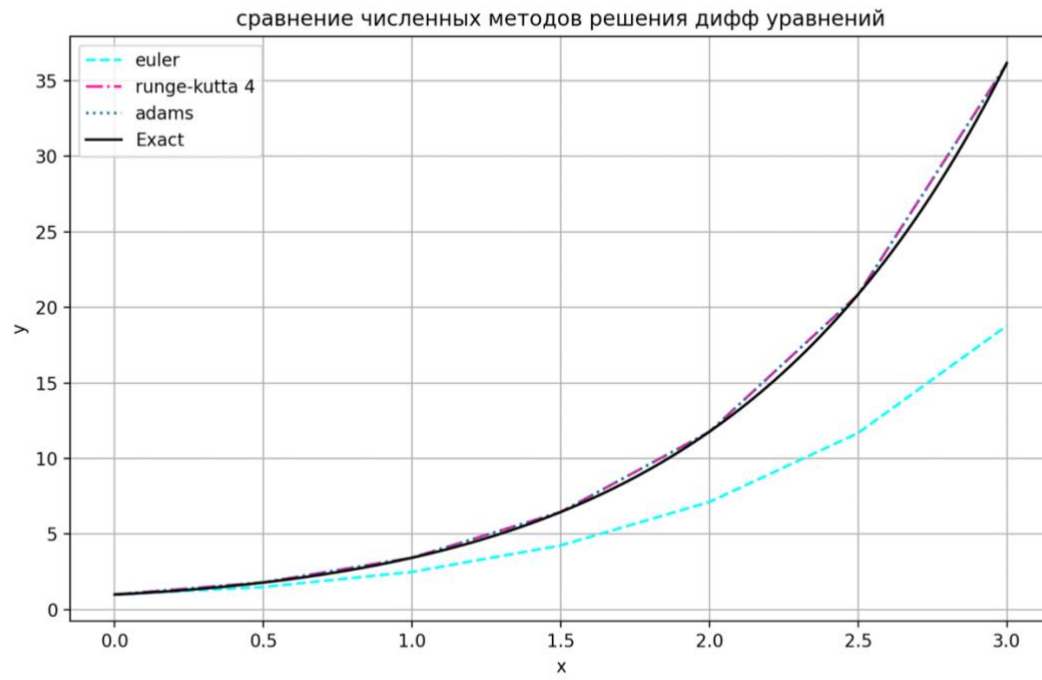
ПРИМЕР 1

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vychma
выберите уравнение для решения:
1: dy/dx = x + y
2: dy/dx = x^2 + y^2
3: dy/dx = 2x - y
введите номер уравнения (1, 2 или 3): 1
введите начальное значение x (x0): 0
введите начальное значение y (y0): 1
введите конечное значение x (xn): 3
введите шаг интегрирования (h): 0.5
введите желаемую точность (epsilon) 0.01

погрешности методов:
euler: 17.156250
runge_kutta4: 2.288814
adams: 0.061091

X          euler          Runge-Kutta4    adams          Exact
-----
0.0000    1.000000    1.000000    1.000000    1.000000
0.5000    1.500000    1.796875    1.796875    1.797443
1.0000    2.500000    3.434692    3.434692    3.436564
1.5000    4.250000    6.458751    6.458751    6.463378
2.0000    7.125000    11.767941   11.765654    11.778112
2.5000    11.687500   20.844027   20.836144    20.864988
3.0000    18.781250   36.129607   36.109983    36.171074
|
```

Figure 1

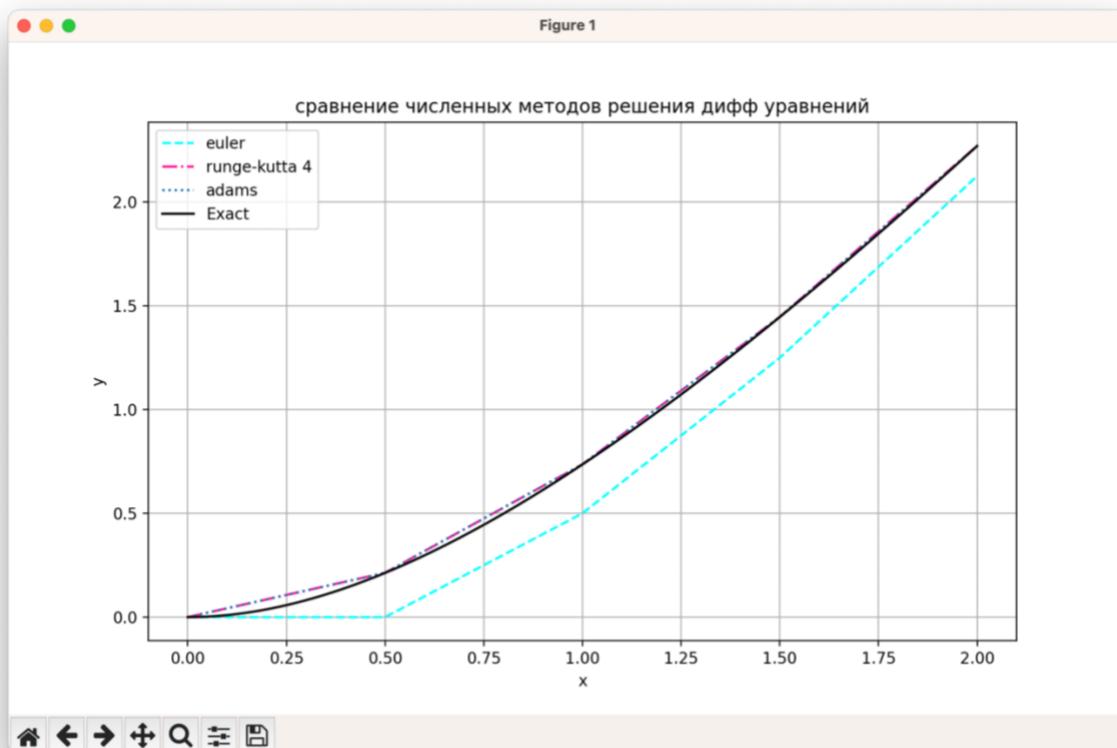


ПРИМЕР 2

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/Pychar
выберите уравнение для решения:
1: dy/dx = x + y
2: dy/dx = x^2 + y^2
3: dy/dx = 2x - y
введите номер уравнения (1, 2 или 3): 3
введите начальное значение x (x0): 0
введите начальное значение y (y0): 0
введите конечное значение x (xn): 2
введите шаг интегрирования (h): 0.5
введите желаемую точность (epsilon) 0.01

погрешности методов:
euler: 2.000000
runge_kutta4: 0.137201
adams: 0.001719
```

X	euler	Runge-Kutta4	adams	Exact
0.0000	0.000000	0.000000	0.000000	0.000000
0.5000	0.000000	0.213542	0.213542	0.213061
1.0000	0.500000	0.736342	0.736342	0.735759
1.5000	1.250000	1.446791	1.446791	1.446260
2.0000	2.125000	2.271100	2.268952	2.270671



Вывод

В результате выполнения лабораторной работы номер 6 я изучила методы решения дифф уравнений, а также закрыла этот предмет, спасибо большое, Екатерина Алексеевна!